The ML System Design Rubric

Scaler School of Technology | ML System Design and LLMOps | Week 7 — Framework Reference

This rubric is a **structured 10-step framework** for answering any ML system design question. Follow the steps in order. Interviewers evaluate your answer on **structure, completeness, and ability to articulate trade-offs** — not on proposing the most complex model.

1 Clarify the Problem

Ask questions before designing. Define scope, constraints, and success criteria. **Key questions:** What must the system achieve? What metrics define success? Are there time, data, or resource constraints?

2 Context

Provide domain and business context. Why does this system exist? What is the business objective? Who are the users? What happens if the system fails?

3 Problem Statement

Formally define the ML problem. Frame it precisely: what is the input, what is the output, and what does the model need to predict or rank?

4 Define Requirements

Split into two categories. **Functional:** what the system must do. **Non-functional:** how it must behave (latency SLA, scale, availability, privacy, maintainability).

5 Estimations

Back-of-envelope sizing. Estimate: number of users, request volume (QPS), data volume per day, storage requirements, compute needed. These numbers drive architecture decisions.

6 System Architecture

Design the end-to-end system. Decompose into: **(a) Data Collection** — sources, signals, ingestion patterns; **(b) Data Engineering** — ETL, storage, schema; **(c) Feature Engineering** — features, feature store; **(d) Preprocessing** — candidate generation, data prep; **(e) Model** — algorithm choice, training pipeline, registry; **(f) Post-processing** — re-ranking, filtering, business rules.

7 Evaluation: Offline and Online

Define how you measure success. **Offline:** precision@K, recall@K, NDCG, AUC-ROC on held-out data. **Online:** north-star metric. Define guardrails (what must *not* degrade).

8 A/B Testing

Design the experiment to validate the model in production. Define: treatment and control groups, sample size, duration, statistical significance threshold, and how you will detect bias.

9 Deployment and Monitoring

Describe the production lifecycle. **Pipelines:** orchestration (Airflow, Kubeflow). **Serving API:** REST/gRPC. **Model controller:** version management, blue-green rollout, rollback. **Dashboards:** latency, drift, business metrics.

10 Risk Management and Trade-offs

Identify what can go wrong and what you gave up. Address: data bias and fairness, model interpretability vs. accuracy, staleness vs. cost, failure modes, and ethical implications.

How to use this rubric: Work through Steps 1–3 aloud with the interviewer. Spend most time on Steps 4–6. Steps 7–10 differentiate senior candidates. Justify every decision: "I chose X over Y because Z. The cost is ..."

Step 1 | Clarify the Problem

Given problem: Design a RAG system for a 10,000-employee company's internal knowledge base with 500,000 documents (Confluence, Jira, Slack, Google Drive, SharePoint).

Clarifying questions an interviewer would expect:

- What document types must be supported? (*Confluence pages, Jira tickets, Slack threads, PDFs, Google Docs*)
- Is role-based access control (RBAC) required? (*Yes — employees only see docs they are authorised for*)
- Is the LLM internal or external API? (*External API acceptable, but consider data sensitivity*)
- What happens when a user asks about a document they cannot access? (*Graceful: "I found relevant information but cannot share it due to access restrictions"*)
- What is the freshness SLA? (*New documents indexed within 30 minutes*)

Step 2 & 3 | Context and Problem Statement

Context: Enterprise productivity tool automating internal knowledge retrieval. Wrong answers with false citations erode trust permanently and employees stop using the system. Reliability must feel like Google Search internally — answers must be verifiable.

Problem Statement: Given an employee's natural language query and their access credentials, retrieve the top-K authorised documents from a 500K internal corpus and generate a cited answer. Input: query + user identity. Output: LLM-generated answer with source citations. Constraints: p99 $<$ 3s, RBAC-correct results only, citations traceable to source URL and paragraph.

Step 4 | Define Requirements

Requirement	Implication for design
Functional	
Multi-source ingestion	Connectors per source (Confluence API, Jira REST, Drive API, Slack Events)
RBAC enforcement	Access control at retrieval time — pre-filter, not post-filter
Citation tracking	Source URL + paragraph stored with every chunk
30-min freshness	Streaming ingestion via Kafka — batch is insufficient
Human feedback loop	Thumbs up/down + optional text; logged for evaluation
Non-functional	
p99 latency $<$ 3s	LLM in serving path — must budget carefully
500 QPS	1,500 concurrent requests at 3s p99
RBAC correctness	Must be 100% — a leak is a security incident
GDPR compliance	Right-to-deletion pipeline required

- **Corpus:** 500K docs × avg 5 chunks = **2.5M chunks**
- **Dense index:** 2.5M × 1536 dims × 4 bytes ≈ **15 GB**
- **BM25 index:** sparse, ≈ **5 GB**
- **Serving:** 500 QPS × 3s = **1,500 concurrent requests**
- **LLM cost:** GPT-4o-mini at 2K tokens/query ≈ $0.0003/query × 500 QPS = **$150/day**
- **Ingestion rate:** 30-min SLA; connectors poll every 5 min
- **Storage:** 500K docs × 50KB avg = **25 GB raw**

Step 6 | System Architecture

The key architectural insight: Enterprise RAG is NOT the same as public search. The three dominant constraints are: **(1) RBAC** — every retrieval must respect permissions; **(2) citation integrity** — users must verify answers; **(3) document diversity** — structured (Jira) and unstructured (Confluence) sources require different chunking. Apply access control at the vector DB level — *pre-filter, never post-filter*.

6a Data Collection

Sources: Confluence (REST API), Jira (REST API), Google Drive (Drive API), Slack (Events API + export), SharePoint (Graph API). Permission metadata (required_roles[]) fetched alongside content at ingestion time.

Ingestion pattern: Webhook-based real-time updates where available (Confluence, Slack); polling every 5 min for others. Kafka as the event bus. Change detection via document hash — only re-index changed docs.

6b & 6c Data Engineering & Feature Engineering

Chunking strategy (source-specific): Confluence: split by section headers (H2/H3). Jira: ticket title + description + comments as one chunk. Slack: conversation threads. PDFs: 512-token sliding window with 50 tokens overlap.

Metadata per chunk: source, URL, author, timestamp, required_roles[], content_hash.

Embedding model: text-embedding-3-small or E5-large-instruct. **Critical:** indexing and serving must use the *identical* model version. Version mismatch causes silent retrieval quality collapse.

BM25 index (Elasticsearch): keyword matching, especially important for exact technical terms (function names, error codes, Jira ticket IDs).

6d & 6e Preprocessing & Models

Retrieval pipeline: (1) Fetch user's role set from IAM. (2) **Pre-filter** vector DB by required_roles[] before ANN search. (3) Dense ANN (FAISS or Pinecone) on filtered space. (4) BM25 keyword search (filtered by roles). (5) RRF (Reciprocal Rank Fusion) merge. (6) Cross-encoder reranker (ms-marco-MiniLM) on top-20. Return top-5 to LLM.

Query classifier: Fast intent detection (DistilBERT, fine-tuned) routes: factual → retrieval + LLM; navigational → direct link; exploratory → multi-hop retrieval.

LLM: GPT-4o-mini with system prompt instructing citation-grounded answers only. Function calling for citation extraction.

6f Post-processing & Serving

Answer generation: LLM generates answer grounded strictly in retrieved chunks. Citations injected as [Source: URL, paragraph N].

Validation: Faithfulness check (is every factual claim in the answer supported by a retrieved chunk?). PII detection before logging. Content safety filter.

Latency budget (p99 = 3s): Role lookup 50ms + Hybrid retrieval 200ms + Reranking 150ms + LLM generation 2,200ms + Response streaming 400ms = **3,000ms**.

Caching: LLM answers cached by query hash (TTL 1h for frequently asked questions). Cache invalidated when source document changes.

Step 7 | Evaluation: Offline and Online

Type	Metric	Why it matters
Offline	Recall@5	Is the answer chunk in top-5 retrieved?
	Citation accuracy	Does cited chunk actually support the answer?
	RAGAS faithfulness	Is every claim in the answer grounded in context?
Online	**Answer satisfaction rate**	Thumbs up % — primary north-star
	Follow-up question rate	User had to rephrase = answer was insufficient
	Zero-result rate	Corpus coverage gap

Critical guardrails: (1) RBAC correctness must be 100% — any access control failure is a security incident, not a quality metric. (2) Hallucination rate $< 2\%$ (spot-checked weekly by human eval team). (3) PII leakage detection must fire on every response.

Step 8 | A/B Testing

Experiment design:

- **Unit:** User session (not individual query — users have session context)
- **Treatment vs. control:** New reranker / new LLM vs. current production
- **Duration:** 14 days minimum — enterprise usage patterns have weekly cycles (lower Fri/Mon)
- **Bias to watch:** Query distribution shift (if a new team onboards, their queries are out-of-distribution). Corpus coverage bias (teams with more indexed content see better results — don't mix them).
- **Decision rule:** Promote if satisfaction rate improves $\geq 2\%$ and RBAC correctness remains 100%. Rollback immediately if any access control failure detected.

Step 9 | Deployment and Monitoring

Ingestion pipeline

- Kafka $\rightarrow$ chunking workers $\rightarrow$ embedding workers $\rightarrow$ vector DB upsert
- Target: < 30 min from doc creation to searchable
- Right-to-deletion: doc deleted in source $\rightarrow$ all chunks removed from vector DB within 24h (GDPR)
- Permission sync: when roles change in IAM, update chunk metadata within 5 min

Serving

- Stateless query pods; session context in Redis
- Circuit breaker on LLM API — if timeout, return top-K chunks without LLM summary
- Blue-green for new embedding models (requires full reindex — expensive, plan carefully)
- Canary for LLM model updates (no reindex needed)

6 Key Metrics to Monitor:

1. **Index freshness by source** — minutes since last update per connector
2. **RBAC permission sync lag** — time from IAM change to chunk metadata update
3. **LLM API latency p99** — alert if > 2.5s
4. **Answer satisfaction rate** — 7-day rolling; alert on $>5\%$ drop
5. **Cache hit rate** — low = cost spike
6. **Zero-result rate by department** — identifies corpus coverage gaps

Step 10 | Risk Management and Trade-offs

Decision	Choice	Alternative	Why this choice
RBAC enforcement	Pre-filter at vector DB	Post-filter after retrieval	Post-filter can leak document titles/snippets. Pre-filter is the only safe option.
LLM provider	External API (GPT-4o-mini)	Self-hosted open-source	External = simpler ops. Risk: enterprise data leaves the org. Mitigate with data anonymisation or choose self-hosted for sensitive content.
Embedding model	Shared model for all sources	Per-source fine-tuned models	Shared model is simpler; one reindex when upgrading. Per-source better quality but 5× more ops complexity.
Chunk size	512 tokens, 50 overlap	Smaller (128t) or larger (1024t)	Too small = lost context, noisy retrieval, more RBAC checks. Too large = irrelevant content in LLM prompt. 512 is the empirical sweet spot.
Cache strategy	Query-hash TTL cache	No caching	Common enterprise queries ("What is our leave policy?") are asked 100s of times. Cache reduces LLM cost 40–60%. Invalidate on source change.

Failure Modes and Recovery

What breaks	How to detect	Recovery
Embedding model version mismatch	Retrieval quality collapses silently; satisfaction rate drops without latency change	Version-lock embedding model in chunk metadata; reject queries if serving model $\neq$ index model
LLM API timeout	p99 latency spike; circuit breaker fires	Return top-K chunks without LLM summary with message "Showing relevant documents — summary unavailable"
Permission sync lag	User sees doc they should no longer access	Re-check permissions at query time (not just index time); maintain permission version in chunk metadata
GDPR deletion SLA breach	Deletion pipeline alert fires; chunk still in index after 24h	Async deletion pipeline with SLA monitoring; escalate to on-call if $>$ 24h
Ingestion backlog $>$ 30 min	Freshness monitor alert; Kafka consumer lag grows	Scale up embedding workers; prioritise high-traffic source connectors